



gRPC Communication Patterns

Cliente / Servidor

72.42 Programación de Objetos Distribuidos





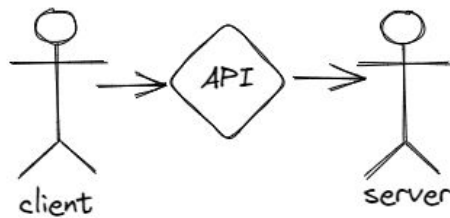
Separación en proyectos

api, client y server



Separación en proyectos

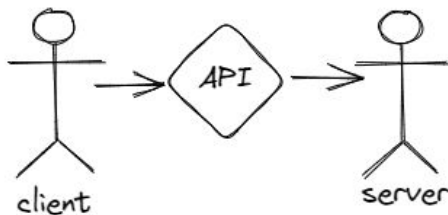
Queremos **separar el código del servicio y del cliente** para simular la situación real donde serían proyectos probablemente mantenidos por equipos diferentes



Por lo cual generamos un proyecto por rol

Separación en proyectos

En gRPC ambos roles/proyectos deben tener la definición de los servicios (el .proto) y generar el código del servicio a partir de esa definición. Sobre esto cada uno genera su código particular



Para no “copy-pastear” generamos un **tercer proyecto (api)** que contenga dicho código y cada uno lo podrá incorporar como dependencia



Maven Assembly

Ejecución del proyecto





Plugin Maven Assembly

La ejecución con el plugin `spring-boot:run` es aceptable para el desarrollo pero en caso de que deseemos distribuir nuestra aplicación es mejor generar un comprimido con todo lo necesario.

El proyecto cuenta con el ***plugin maven-assembly***, que permite empaquetar proyectos con sus dependencias en un `tar.gz`.

A medida que se va generando el código para el proyecto se ejecuta:

```
$> ./mvnw clean package
```

para construir el proyecto, correr *tests* y empaquetar.

Genera un archivo **`tar.gz`** con las clases y jars de dependencias listo para usar, uno por cada proyecto.



Plugin Maven Assembly

Tanto en server como en client, en el directorio
src/main/assembly/overlay

se encuentran scripts (.sh) que se incluirán en el empaquetado final (.tar.gz)

Por ejemplo, para el caso de run-client.sh

```
#!/bin/bash

PATH_TO_CODE_BASE=`pwd`

MAIN_CLASS="ar.edu.itba.pod.grpc.client.Client"

java $JAVA_OPTS -cp 'lib/jars/*' $MAIN_CLASS $*
```



Plugin Maven Assembly

Para ejecutar los proyectos **server** y **client**

1. Entrar en la carpeta **target** de cada proyecto
 2. Descomprimir los ***-bin.tar.gz** (usando `tar -xzf <path>`)
 3. Entrar en la carpeta que aparece y dar permisos de ejecución a los scripts **run-*.sh** (`chmod u+x <path>`)
 4. Revisar dentro de los scripts que el Fully Qualified Name de la clase a ejecutar sea el correcto
 5. Una vez hechos estos pasos, se puede ejecutar en 2 consolas diferentes
- 

Ejercicio 1 - gRPC Greeter - Assembly

- Retomar el proyecto de la clase anterior
- Siguiendo las instrucciones del **README** en la sección **Shell**, **ejecutar** en dos terminales distintas:
 - **Server**
 - **Client**
- Verificar que obtiene el resultado esperado



Docker

Ejecución del proyecto



Docker

Otra forma de ejecutar el proyecto es utilizando Docker. Para ello se generará un contenedor para el servidor y un contenedor para el cliente con la particularidad de que este último se crea y se destruye.

Tanto en server como en client, se encuentra un archivo `Dockerfile` con las instrucciones para construir la imagen que se ejecutará en el contenedor.

```
FROM amazoncorretto:25-alpine-jdk
WORKDIR /app
RUN apk add --no-cache bash
COPY target/grpc-greeter-server-2026.1Q-bin.tar.gz .
RUN tar -xzf grpc-greeter-server-2026.1Q-bin.tar.gz \
    && rm grpc-greeter-server-2026.1Q-bin.tar.gz \
    && chmod +x grpc-greeter-server-2026.1Q/run-server.sh
WORKDIR /app/grpc-greeter-server-2026.1Q
EXPOSE 50051
ENTRYPOINT ["bash", "run-server.sh"]
```

Docker Compose

- Con el archivo `compose.yml` se simplifica la configuración y ejecución del proyecto.
- Notar que el módulo `client` tiene un `profile` para evitar que ese contenedor se ejecute por defecto
- Se define un `.env` para `host` y `puerto`

```
$> docker compose up --build
```

```
$> docker compose --profile client run  
--rm client
```

```
services:  
  server:  
    container_name: greeter_server  
    image: greeter_server:latest  
    build:  
      context: ./server  
    ports:  
      - "${GRPC_PORT}:${GRPC_PORT}"  
  client:  
    container_name: greeter_client  
    image: greeter_client:latest  
    build:  
      context: ./client  
    profiles: ["client"]  
    depends_on:  
      - server  
    environment:  
      SPRING_GRPC_CLIENT_CHANNELS_LOCAL_ADDRESS:  
        "server:${GRPC_PORT}"
```

Ejercicio 2 - gRPC Greeter - Docker

- Siguiendo las instrucciones del **README** en la sección Docker, **ejecutar**, en dos terminales distintas:
 - **Server**
 - **Client**
- Verificar que obtiene el resultado esperado



Deadlines

gRPC



gRPC - Deadlines - Client

- Un cliente puede definir un deadline (*timeout*) que indica hasta cuándo está dispuesto a esperar.
- Por defecto gRPC no define un deadline por lo que un cliente podría quedarse esperando una respuesta para siempre.
- Si se vence el plazo, el cliente aborta y se obtiene DEADLINE_EXCEEDED

```
try {  
    var response = greeterBlockingStub  
        .withDeadlineAfter(5, TimeUnit.SECONDS)  
        .sayHello(request);  
    log.info("Greeting: {}", response.getMessage());  
} catch (StatusRuntimeException ex) {  
    log.error(ex.getMessage());  
}
```

```
DEADLINE_EXCEEDED: CallOptions deadline exceeded after  
4.989594083s. Name resolution delay 0.004798625 seconds.
```

gRPC - Deadlines - Server

- Para evitar desperdiciar recursos gRPC cancela automáticamente la ejecución del método cuando el plazo se vence y gracias a eso se obtiene CANCELLED en el cliente.
- Pero esa cancelación no detiene por sí sola el procesamiento interno de la aplicación. Si se dispararon tareas para atender esa invocación, se puede verificar periódicamente si la llamada fue cancelada y así finalizar el procesamiento

```
if (Context.current().isCancelled()) {  
    responseObserver.onError(...);  
    return;  
}
```



Health

Spring gRPC



gRPC - Health - Server

gRPC ofrece por defecto un servicio para realizar un *health-check* (El .proto ya está definido). Con Spring gRPC basta con crear el @Bean de HealthStatusManager

```
@Configuration
public class HealthConfig {

    @Bean
    HealthStatusManager healthStatusManager() {
        var healthStatusManager = new HealthStatusManager();
        healthStatusManager.setStatus(GreeterGrpc.SERVICE_NAME,
                                     HealthCheckResponse.ServingStatus.SERVING);
        return healthStatusManager;
    }
}
```

gRPC - Health - Servant

Los estados posibles son SERVING, NOT_SERVING y UNKNOWN (el *default*)

```
@Service
public class Servant extends GreeterGrpc.GreeterImplBase {
    private final HealthStateManager healthStateManager;
    public Servant(HealthStateManager healthStateManager) {
        this.healthStateManager = healthStateManager;
    }
    @Override
    public void sayHello(HelloRequest request, StreamObserver<HelloReply> responseObserver) {
        // ...
        if (...) {
            healthStateManager.setStatus(GreeterGrpc.SERVICE_NAME,
                                         HealthCheckResponse.ServingStatus.NOT_SERVING);
        }
        // ...
    }
}
```

gRPC - Health - Client

Del lado del cliente se configura para que se realice un *health check* automático.

```
spring:
  grpc:
    client:
      channels:
        local:
          address: localhost:50051
          health:
            enabled: true
            service-name: Greeter
logging:
  level:
    root: info
```

```
<dependencies>
...
<dependency>
  <groupId>io.grpc</groupId>
  <artifactId>grpc-services</artifactId>
</dependency>
</dependencies>
```

UNAVAILABLE: Health-check service responded NOT_SERVING for 'Greeter'



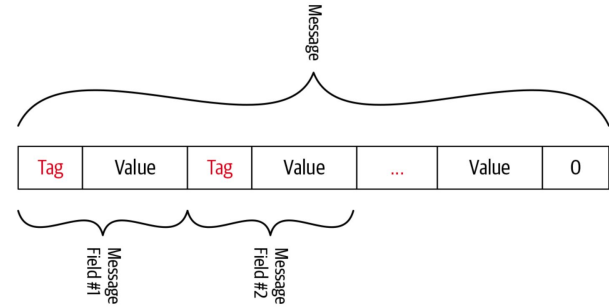
Protocol Buffers

Language Guide



Protocol Buffers

```
message User {  
  int32 id = 1;  
  string user_name = 2;  
  string display_name = 3;  
  AccountStatus status = 4;  
  repeated string preferences = 5;  
}
```



- Las estructuras pueden ser message o service
- Los tipos pueden ser
 - Scalar Value: double, float, int32, uint64, bool, string, etc.
 - Enumerations
 - Otros message ya definidos (composición)
- Es obligatorio asignar a cada campo un número único entre todos los campos de ese mensaje

Protocol Buffers

- En cuanto a la cardinalidad existen los modificadores:
 - **optional**: Ese campo tendrá dos estados posibles, seteado y contiene un valor que será serializado o “unset” de forma que retorna el valor por defecto y no será serializado
 - **repeated**: El campo se repite 0 o más veces y se preserva el orden (lista)
 - **map**: Una colección de pares clave/valor
- Se permiten Nested Types
- Los comentarios soportados son:
 - `// Comentario Inline`
 - `/* Comentario Multiline */`

```
message UserRoles {  
    map<string, Role> roles_by_site = 1;  
}
```

Protocol Buffer - Enum

Protobuf permite usar **enum** como tipo para definir uno con un número finito de posibles valores

```
enum AccountStatus {  
    ACCOUNT_STATUS_UNSPECIFIED = 0;  
    ACCOUNT_STATUS_PENDING = 1;  
    ACCOUNT_STATUS_ACTIVE = 2;  
    ACCOUNT_STATUS_SUSPENDED = 3;  
    ACCOUNT_STATUS_CLOSED = 4;  
}
```

Los valores deben contar con el prefijo del enum (ACCOUNT_STATUS_...) y es necesario declarar un default value llamado ..._UNSPECIFIED con 0 como valor numérico

Protocol Buffer - Servicio

El siguiente servicio cuenta con dos métodos:

```
service UserService {  
  rpc DoLogin(LoginInformation) returns (User);  
  rpc GetRoles(User) returns (UserRoles);  
}  
  
message LoginInformation { ... }  
  
message User { ... }  
  
message UserRoles { ... }
```

gRPC - Wrappers

Para no estar creando mensajes vacíos o simples que cuentan con un solo tipo escalar se provee una biblioteca de mensajes *wrappers*:

- [google/protobuf/wrappers.proto](#)
- [google/protobuf/empty.proto](#)

```
message StringValue {  
  // The string value.  
  string value = 1;  
}
```

```
message Empty {}
```

Para usarlos basta con incluir la sentencia import correspondiente:

```
import "google/protobuf/wrappers.proto";  
import "google/protobuf/empty.proto";  
service PingService {  
  rpc Ping(google.protobuf.Empty) returns (google.protobuf.StringValue);  
}
```

Ejercicio 3: gRPC User

- **Clonar el repo usando el link de GitHub Classroom (Campus)**
- Abrirlo con un IDE
- Leer el **README.md**
- **Implementar los métodos `DoLogin` y `GetRoles`**
del `UserService` utilizando los datos en memoria de `passwordsByUser`
- Realizar una prueba completa



Llamados Asincrónicos

FutureStub y Stub





gRPC - Llamados Asincrónicos

Para situaciones donde queremos utilizar llamados no bloqueantes gRPC provee dos estrategias de llamados asincrónicos:

- Resolución por `Future`
- Resolución por `StreamObserver` / `Callback`

De esta manera podemos aprovechar las ventajas de realizar llamados asincrónicos y liberar los recursos del sistema mientras el servidor procesa el pedido.

En principio para usar llamadas asincrónicas el código en el servidor no varía, si cambia el cliente.



gRPC - FutureStub

El cliente instancia un nuevo *stub* llamado ***FutureStub** a partir del método `newFutureStub` del código generado

```
@Bean
UserServiceGrpc.UserServiceFutureStub userFutureStub(GrpcChannelFactory channels) {
    return UserServiceGrpc.newFutureStub(channels.createChannel("local"));
}
```

gRPC - FutureStub

Al realizar un llamado a los métodos remotos del **FutureStub** el mismo retorna un **ListenableFuture**

```
public interface ListenableFuture<V extends @Nullable Object>  
                                extends Future<V> {  
    void addListener(Runnable listener, Executor executor);  
}
```

Por lo que permite acceder a la respuesta de dos maneras:

- Utilizando la funcionalidad de la interfaz Future
- Agregando un *callback*

```
Futures.addCallback(userFuture, userCallback, executorService);
```

gRPC - FutureStub

El callback es un `FutureCallback` que provee un método para el caso de error y otro para el *success*.

```
static class UserCallback implements FutureCallback<User> {  
    private static final Logger logger =  
        LoggerFactory.getLogger(UserCallback.class);  
  
    @Override  
    public void onSuccess(User user) {  
        logger.info("received user {}", user);  
    }  
  
    @Override  
    public void onFailure(Throwable throwable) {  
        logger.error("reaching for user", throwable);  
    }  
}
```

Ejercicio 4: gRPC User con Future Stub

- Agregar un nuevo método al cliente que solicite los roles utilizando un ***callback*** que loguee los mismos en consola. Puede utilizar un `await` sobre un `CountDownLatch` y un `Executors.newSingleThreadExecutor` para el `ExecutorService` necesario
- Realizar una prueba completa

gRPC - Async Stub

La estrategia alternativa para llamados asincrónicos es utilizar el *Stub*

```
@Bean
UserServiceGrpc.UserServiceStub userAsyncStub(GrpcChannelFactory channels) {
    return UserServiceGrpc.newStub(channels.createChannel("local"));
}
```

En este caso los métodos definidos por el servicio además del parámetro de entrada reciben un segundo parámetro de tipo **StreamObserver** que funciona como *callback*

```
userServiceStub.doLogin(loginInformation, observer);
```

Ejercicio 5: gRPC User con Async Stub

- Agregar un nuevo método al cliente que solicite los roles utilizando un **StreamObserver** que loguee los mismos en consola. Puede utilizar un `await` sobre un `CountDownLatch`
- Realizar una prueba completa



CREDITS

Content of the slides:

- » POD - ITBA

Images:

- » POD - ITBA
- » gRPC Up & Running: Indrasiri, Kuruppu (2020) O'Reilly Media
- » Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- » Presentation template by [SlidesCarnival](https://slidescarnival.com)
 - » Photographs by [Unsplash](https://unsplash.com)
- 